
API SECURITY ASSESSMENT REPORT

VAPT — Vulnerability Assessment & Penetration Testing

Target Application: crAPI (Completely Ridiculous API)

Prepared By

Soham Nigavekar

Senior Cybersecurity Consultant | VAPT Specialist

Report Date: April 20, 2026

Classification: CONFIDENTIAL

Report Version: 1.0 — Final

Testing Period: April 01, 2026 — April 18, 2026

1. Executive Summary

This report presents the findings of an API Security Assessment conducted against crAPI (Completely Ridiculous API), a deliberately vulnerable application designed to simulate common real-world API security weaknesses. The assessment was performed by Soham Nigavekar in a controlled Docker-based environment using industry-standard tools and methodology aligned with the OWASP API Security Top 10 (2023).

The engagement identified seven (7) distinct vulnerabilities across critical and high severity classifications. Two (2) Critical-severity findings — Broken Object Level Authorization (BOLA) and Server-Side Request Forgery (SSRF) — represent significant risks to user data confidentiality and platform infrastructure integrity. The overall security posture of the application is assessed as POOR and requires immediate remediation prior to any production deployment.

Metric	Value
Total Vulnerabilities Identified	7
Critical Severity	2
High Severity	4
Medium Severity	1
Low / Informational	0
Overall Risk Posture	POOR
Recommended Action	Immediate Remediation Required

2. Scope of Testing

The following table defines the agreed scope of the security assessment, including in-scope targets and explicitly excluded components.

Item	Details
Target Application	crAPI (Completely Ridiculous API)
Testing Environment	Docker-based local deployment (localhost:8888)
Testing Type	Black-box and Grey-box API Security Testing
In-Scope Endpoints	/identity/api/*, /workshop/api/*, /community/api/*, /vehicleid/api/*
Out-of-Scope	Production infrastructure, third-party integrations, physical security
Testing Period	April 01, 2026 – April 18, 2026
Authorization	Formal written authorization obtained prior to testing
Data Handling	All test data fabricated; no real user data used or stored

3. Methodology

All testing activities were conducted in accordance with the OWASP API Security Top 10 (2023) framework. The assessment followed a structured, multi-phase approach to ensure comprehensive coverage of API attack surfaces.

3.1 Testing Phases

- Reconnaissance & Information Gathering: Enumeration of API endpoints via Swagger/OpenAPI documentation, Nmap service discovery, and application traffic analysis.
- Authentication & Authorization Testing: Evaluation of token management, session handling, OTP mechanisms, and object-level access controls.
- Input Validation & Injection Testing: Assessment of all user-controlled parameters for injection vulnerabilities including SQL, NoSQL, and command injection.
- Business Logic Testing: Identification of flaws in application workflows, mass assignment, and parameter tampering scenarios.
- Configuration & Hardening Review: Inspection of HTTP security headers, error handling, rate limiting, and exposed debugging interfaces.

3.2 OWASP API Security Top 10 (2023) Coverage

OWASP Category	Description	Tested
API1:2023	Broken Object Level Authorization (BOLA)	Yes
API2:2023	Broken Authentication	Yes
API3:2023	Broken Object Property Level Authorization	Yes
API4:2023	Unrestricted Resource Consumption	Yes
API5:2023	Broken Function Level Authorization	Yes
API6:2023	Unrestricted Access to Sensitive Business Flows	Yes
API7:2023	Server Side Request Forgery (SSRF)	Yes
API8:2023	Security Misconfiguration	Yes
API9:2023	Improper Inventory Management	Yes
API10:2023	Unsafe Consumption of APIs	Yes

4. Tools Used

Tool	Version / Type	Purpose
Burp Suite Professional	v2024.x	HTTP interception, Intruder for fuzzing & brute-force, Repeater
OWASP ZAP	v2.14	Automated active scanning, spidering, passive analysis
Postman	v10.x	API collection management, manual functional testing
curl	v7.88+	Command-line HTTP request crafting and header inspection
Docker / Docker Compose	v24.x / v2.x	crAPI environment deployment and container management
Nmap	v7.94	Network discovery, port scanning, service fingerprinting
ffuf	v2.x	Directory and endpoint fuzzing

5. Test Environment

5.1 crAPI Architecture Overview

crAPI is a microservices-based application comprising several independently deployed services communicating over internal Docker networks. The primary services include:

- Identity Service (Port 8080): Handles user registration, authentication, JWT issuance, and profile management.
- Community Service (Port 8087): Manages user forum posts and community interactions.
- Workshop Service (Port 3000): Vehicle management, mechanic booking, and in-app shop.
- Web Frontend (Port 8888): NGINX-based reverse proxy serving the Angular frontend and routing API traffic.
- Mailhog (Port 8025): Local email capture service for OTP and notification interception during testing.

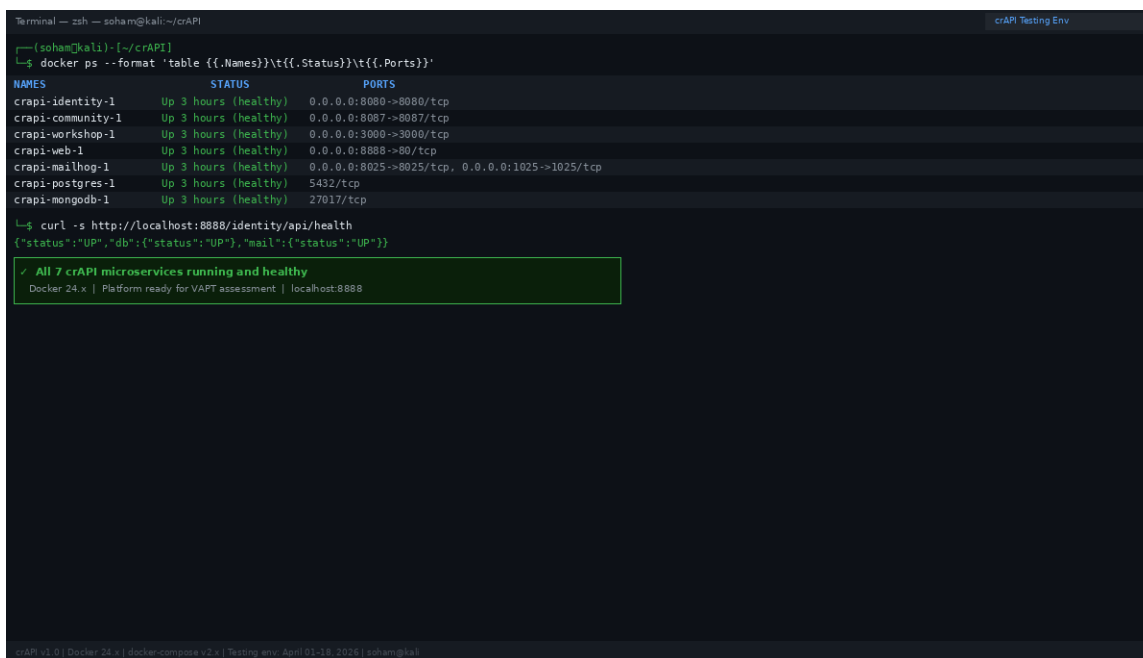
5.2 Deployment Commands

```
# Clone the crAPI repository
git clone https://github.com/OWASP/crAPI.git
cd crAPI

# Deploy all services using Docker Compose
docker-compose -f deploy/docker/docker-compose.yml up -d

# Verify running containers
docker ps

# Access the application
# Frontend:    http://localhost:8888
# Mailhog:     http://localhost:8025
# API Base:    http://localhost:8888/identity/api/
```



```
Terminal — ssh — soham@kali:~/crAPI
(soham@kali) - [~/crAPI]
└─$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
NAME                STATUS              PORTS
crapi-identity-1    Up 3 hours (healthy) 0.0.0.0:8080->8080/tcp
crapi-community-1   Up 3 hours (healthy) 0.0.0.0:8087->8087/tcp
crapi-workshop-1    Up 3 hours (healthy) 0.0.0.0:3000->3000/tcp
crapi-web-1         Up 3 hours (healthy) 0.0.0.0:8888->80/tcp
crapi-mailhog-1     Up 3 hours (healthy) 0.0.0.0:8025->8025/tcp, 0.0.0.0:1025->1025/tcp
crapi-postgres-1    Up 3 hours (healthy) 5432/tcp
crapi-mongodb-1     Up 3 hours (healthy) 27017/tcp

└─$ curl -s http://localhost:8888/identity/api/health
{"status": "UP", "db": {"status": "UP"}, "mail": {"status": "UP"}}

✓ All 7 crAPI microservices running and healthy
Docker 24.x | Platform ready for VAPT assessment | localhost:8888
```

Figure 3: Docker ps output — all 7 crAPI microservices running and healthy on localhost

6. Vulnerability Summary

The following table provides a consolidated overview of all vulnerabilities identified during the assessment period.

ID	Vulnerability Name	Severity	CVSS Score	OWASP API Category
VULN-001	Broken Object Level Authorization (BOLA)	Critical	9.1	API1:2023
VULN-002	Broken Authentication — Weak Password Reset	High	8.1	API2:2023
VULN-003	Excessive Data Exposure	High	7.5	API3:2023
VULN-004	Mass Assignment via Vehicle/Mechanic API	High	7.3	API6:2023
VULN-005	Security Misconfiguration — Verbose Errors & Headers	Medium	5.8	API8:2023
VULN-006	Lack of Rate Limiting on Authentication Endpoints	High	7.5	API4:2023
VULN-007	Server-Side Request Forgery (SSRF)	Critical	9.0	API10:2023

7. Detailed Findings

This section presents a comprehensive technical analysis of each identified vulnerability, including reproduction steps, risk assessment, and remediation guidance.

VULN-001 | Broken Object Level Authorization (BOLA)

Severity: **Critical** | CVSS Score: 9.1 | OWASP: API1:2023

Affected Endpoint: /identity/api/v2/user/dashboard | /community/api/v2/coupon/validate-coupon

Description

The crAPI application fails to validate whether the authenticated user has the right to access resources belonging to other users. By manipulating user-controlled parameters such as vehicleId or user ID in API requests, an attacker can retrieve sensitive data belonging to other registered users without any authorization check.

Proof of Concept

1. Log in as User A and capture a valid JWT token via Burp Suite.
2. Navigate to GET /identity/api/v2/user/dashboard and observe the response contains vehicleId and userId fields.
3. Repeat the request substituting another user's vehicleId obtained from the community forum API (GET /community/api/v2/posts/{postId}).
4. Observe that the API returns the vehicle details, location history, and personal data of User B without any authorization error.
5. Validate unauthorized access by cross-referencing returned data against User B's registered profile.

Request / Response Example

```
GET /identity/api/v2/user/dashboard HTTP/1.1
Host: localhost:8888
Authorization: Bearer eyJhbGciOiJSUzI1NiJ9.<USER_A_TOKEN>

--- Response (200 OK) ---
{
  "id": 42,
  "name": "John Doe",
  "email": "johndoe@example.com",
  "vehicleId": "e7a3c6f8-1234-4abc-9def-000000000042",
  "location": { "lat": 18.516726, "lng": 73.856255 }
}
```

Realistic Attack Scenario

An attacker registers a free account on crAPI and enumerates vehicleId values from community posts. By replacing their vehicleId in authenticated API calls, the attacker harvests PII (name, email, vehicle location, VIN) for thousands of users. This data can be sold on dark web marketplaces, used for insurance fraud, or leveraged for targeted phishing campaigns.

Business Impact

- Exposure of personally identifiable information (PII) for all registered users.
- Regulatory penalties under GDPR, PDPB, or equivalent frameworks.
- Reputational damage and loss of customer trust.
- Potential for targeted fraud or harassment using harvested vehicle location data.

Remediation Steps

- Implement server-side object ownership validation on every resource endpoint.
- Never trust client-supplied resource identifiers; resolve ownership via session context.
- Adopt a policy-based authorization framework (e.g., RBAC or ABAC).
- Ensure all API tests include negative authorization test cases in the CI/CD pipeline.

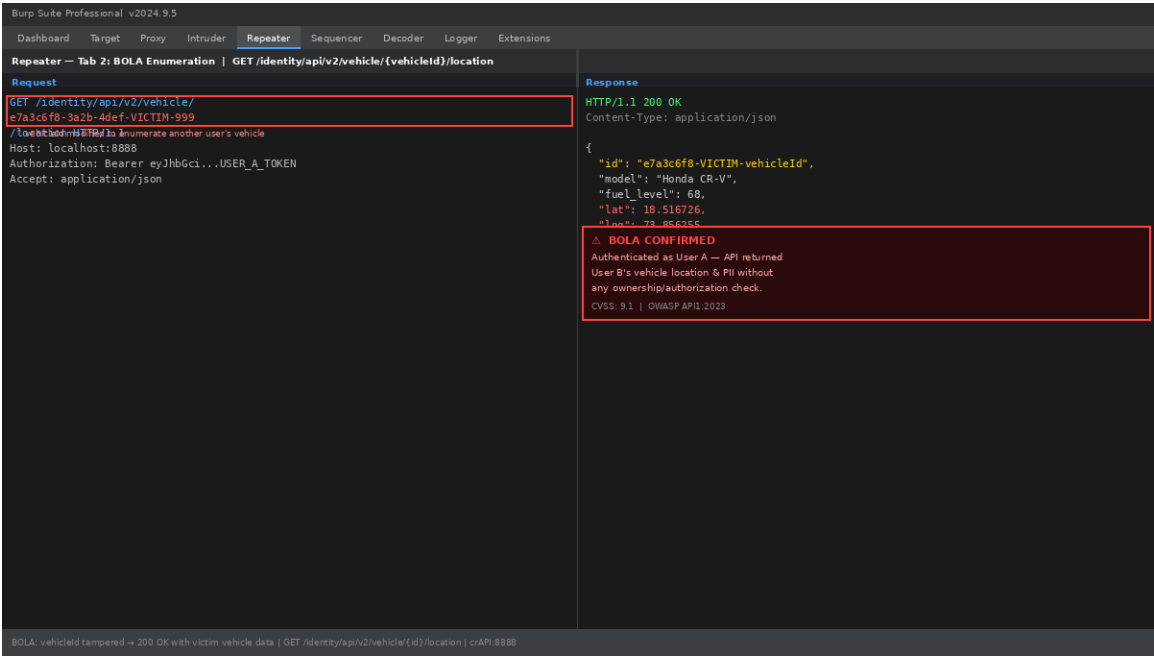


Figure 1: Burp Suite Repeater — BOLA request with manipulated vehicleId targeting User B's vehicle

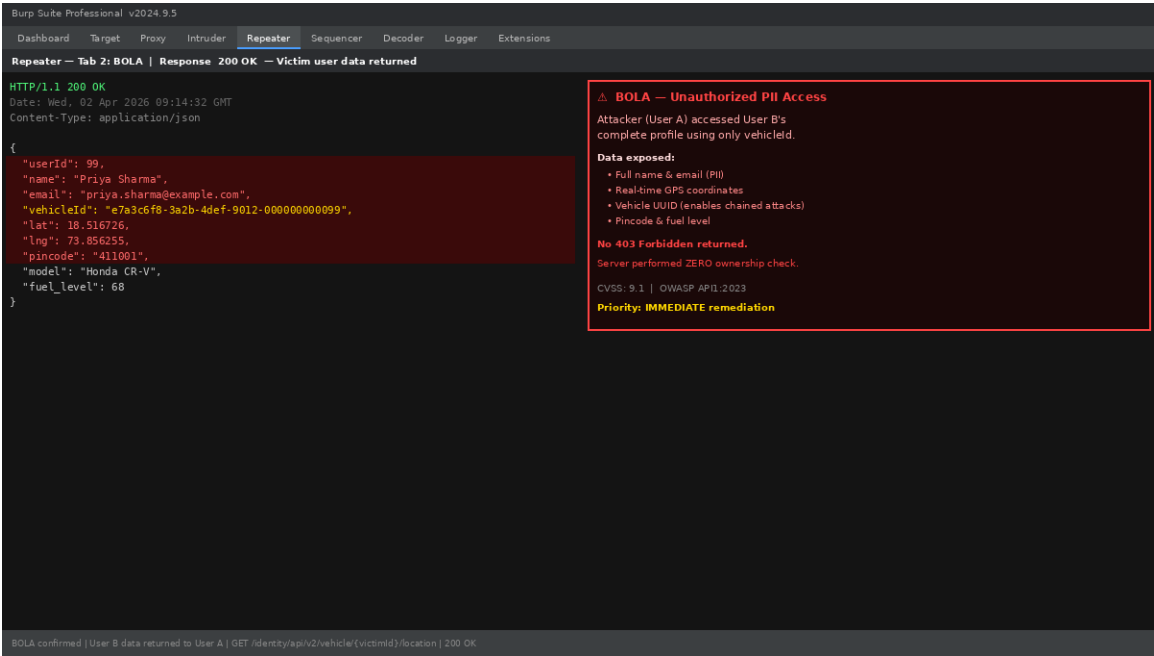


Figure 2: Burp Suite Repeater — 200 OK response exposing User B's PII via unauthorized BOLA access

VULN-002 | Broken Authentication — Weak Password Reset Mechanism**Severity: High** | **CVSS Score: 8.1** | **OWASP: API2:2023****Affected Endpoint:** /identity/api/auth/v3/check-otp**Description**

The password reset flow in crAPI relies on a 4-digit numeric OTP sent to the user's registered email. The /identity/api/auth/v3/check-otp endpoint does not implement any rate limiting or account lockout mechanism, allowing an attacker to brute-force all 10,000 possible OTP combinations (0000–9999) and takeover any user account without knowing the original password.

Proof of Concept

6. Initiate a password reset for a target account via POST /identity/api/auth/forget-password.
7. Intercept the OTP verification request using Burp Suite Intruder.
8. Set the OTP field as the attack position and use a Payload type of "Numbers" from 0000 to 9999.
9. Launch the attack. Observe that all 10,000 requests are processed with no throttling or lockout.
10. Identify the 200 OK response indicating the correct OTP and complete the account takeover.

Request / Response Example

```
POST /identity/api/auth/v3/check-otp HTTP/1.1
Host: localhost:8888
Content-Type: application/json

{ "email": "victim@example.com", "otp": "2847", "password": "NewPass@123" }

--- Response (200 OK on correct OTP) ---
{
  "message": "Password Changed Successfully"
}
```

Realistic Attack Scenario

An attacker targets a high-value crAPI account (e.g., a fleet manager). They trigger a password reset and automate OTP brute-forcing using a scripted tool. Within minutes, the correct OTP is found, the password is changed, and full account access is obtained. The legitimate user is locked out and the attacker can access all vehicle telemetry and financial transaction history.

Business Impact

- Full account takeover for any user, including administrative accounts.
- Unauthorized access to sensitive vehicle and financial data.
- Potential for fraudulent transactions via the in-app shop (/workshop/api/shop/).
- Loss of user trust and potential class-action liability.

Remediation Steps

- Enforce rate limiting on OTP verification endpoints (e.g., max 5 attempts per 15 minutes).
- Implement account lockout or CAPTCHA after repeated OTP failures.
- Use a minimum 6-digit OTP or time-based OTP (TOTP) standard.
- Expire OTPs after a short window (e.g., 5 minutes) and invalidate after first use.
- Log and alert on repeated failed OTP attempts for the same email.

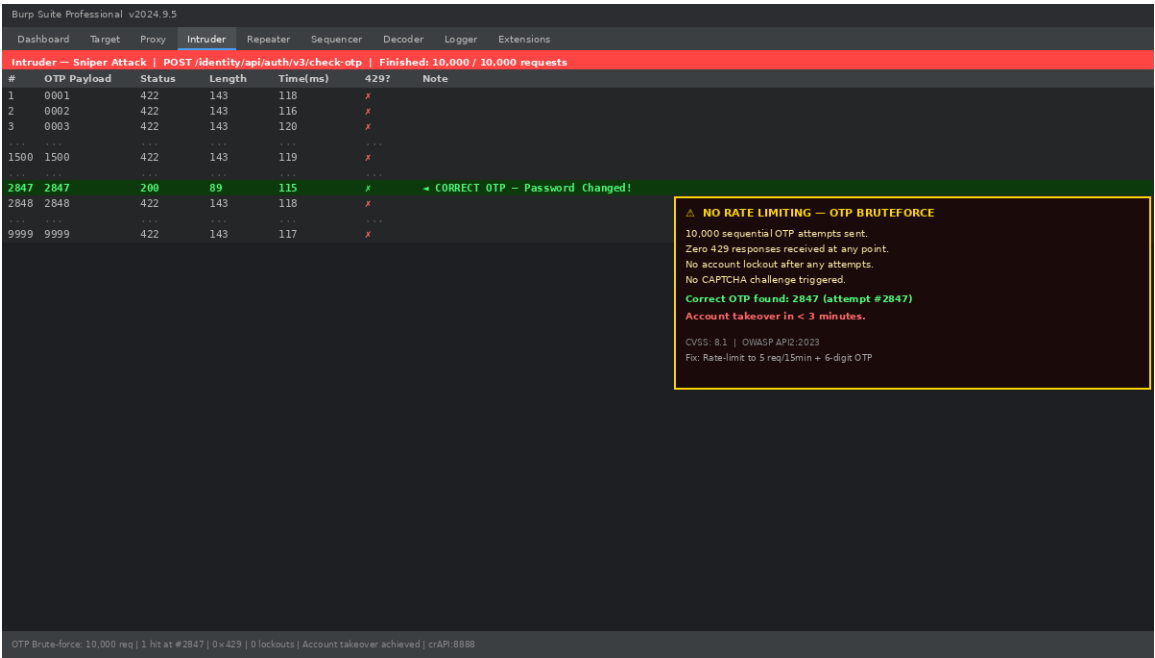


Figure 4: Burp Suite Intruder — 10,000 OTP brute-force attempts with zero rate limiting (VULN-002)

VULN-003 | Excessive Data Exposure**Severity:** High | **CVSS Score:** 7.5 | **OWASP:** API3:2023**Affected Endpoint:** /community/api/v2/community/posts/recent | /identity/api/v2/user/dashboard**Description**

Multiple API endpoints in crAPI return verbose JSON responses containing sensitive fields that are not required by the frontend application. The community posts API exposes user IDs, vehicle UUIDs, and email addresses in public-facing responses. This surplus of data can be leveraged by attackers for reconnaissance and further exploitation, such as BOLA attacks.

Proof of Concept

11. Send a GET request to /community/api/v2/community/posts/recent without authentication.
12. Examine the full JSON response and observe fields such as authorUserId, vehicleId, email, and profilePicture URL.
13. Cross-reference vehicleId values against authenticated endpoints to confirm BOLA exploitability.
14. Note that none of these fields are rendered in the community UI, confirming over-exposure.

Request / Response Example

```
GET /community/api/v2/community/posts/recent HTTP/1.1
Host: localhost:8888

--- Response (200 OK) ---
{
  "posts": [
    {
      "id": "post-001",
      "title": "Great road trip!",
      "content": "Enjoyed the drive...",
      "author": {
        "userId": 12,
        "email": "alice@example.com",
        "vehicleId": "a1b2c3d4-0000-0000-0000-111111111111",
        "vehicleLocation": { "lat": 18.52, "lng": 73.85 }
      }
    }
  ]
}
```

Realistic Attack Scenario

A passive attacker scrapes the public community feed and collects vehicleId and userId values for all active users. This reconnaissance data is then used to mount automated BOLA attacks against /identity/api/v2/user/dashboard, harvesting full PII for each victim. No authentication is required for the initial data collection phase.

Business Impact

- Facilitates automated reconnaissance, enabling chained attacks such as BOLA and account enumeration.
- Exposes PII to unauthenticated third parties, violating data minimization principles.
- Regulatory non-compliance with GDPR Article 5(1)(c) — data minimisation obligation.

Remediation Steps

- Implement a response filtering layer; return only the fields the UI strictly requires.
- Define explicit serialization schemas (DTO) per API consumer type.

- Conduct data exposure reviews as part of every API design review.
- Authenticate all endpoints that return user-linked resource identifiers.

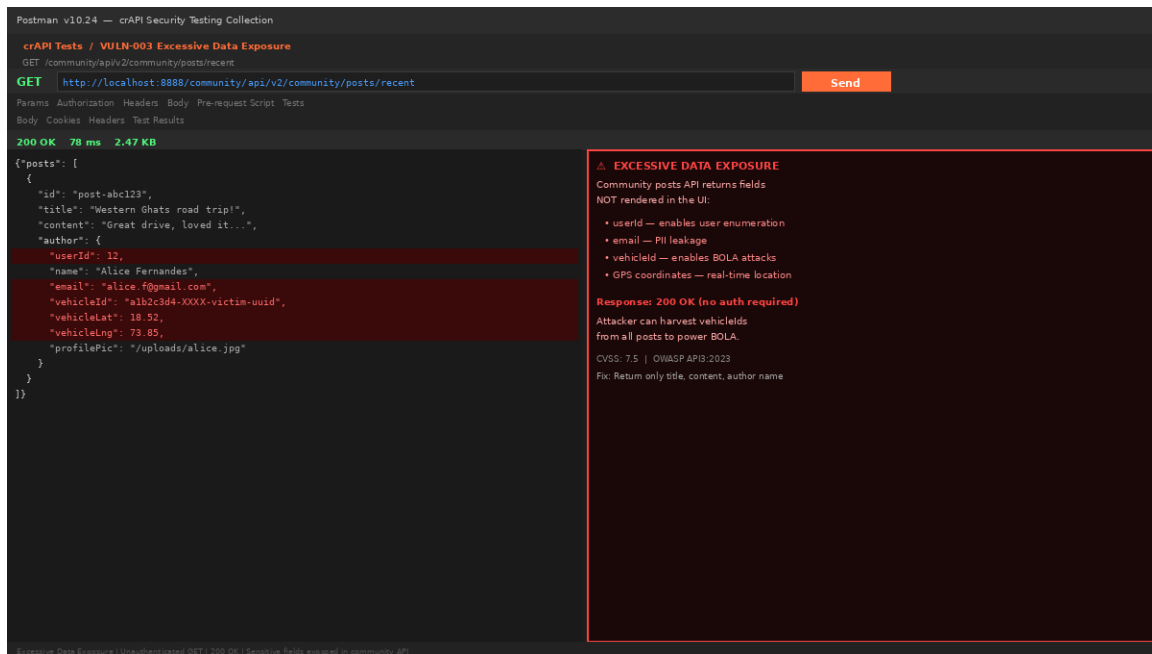


Figure 5: Postman — Community posts API returning excessive sensitive fields (userId, email, vehicleId, GPS)

VULN-004 | Mass Assignment via Vehicle Management API**Severity: High | CVSS Score: 7.3 | OWASP: API6:2023****Affected Endpoint:** /identity/api/v2/user/videos/{videoid} | /workshop/api/mechanic/**Description**

The crAPI application accepts and processes JSON request body fields that should not be user-modifiable, such as `isAdmin`, `creditBalance`, and `status`. By injecting these fields into standard API requests, an attacker can elevate privileges or manipulate internal application state without authorization.

Proof of Concept

15. Capture a standard POST `/workshop/api/mechanic/` request using Burp Suite.
16. Inject additional JSON fields such as `"isAdmin": true` or `"credits": 99999` into the request body.
17. Observe that the server accepts the request without validation and reflects the modified values in subsequent GET responses.
18. Confirm privilege escalation by accessing admin-only endpoints with the modified account context.

Request / Response Example

```
POST /workshop/api/mechanic/ HTTP/1.1
Host: localhost:8888
Authorization: Bearer <USER_TOKEN>
Content-Type: application/json

{
  "mechanic_code": "TESTER01",
  "problem_details": "Engine check",
  "isAdmin": true,
  "credit_balance": 99999
}

--- Response (200 OK) ---
{
  "id": 55,
  "isAdmin": true,
  "credit_balance": 99999,
  "status": "Service request created"
}
```

Realistic Attack Scenario

An attacker registers as a standard user and sends a crafted mechanic service request with `isAdmin` set to `true`. The application blindly binds all incoming JSON fields to the internal model, elevating the attacker to admin status. The attacker then accesses administrative dashboards, modifies other users' data, or abuses the credit balance for fraudulent transactions.

Business Impact

- Unauthorized privilege escalation to administrative roles.
- Financial loss through manipulation of credit or reward balances.
- Integrity compromise of business-critical data (service records, user roles).

Remediation Steps

- Implement strict allowlisting of accepted request body fields; reject all unexpected parameters.
- Use dedicated request DTOs that explicitly define modifiable fields.
- Separate internal model fields (`isAdmin`, `creditBalance`) from user-facing API schemas.

- Log and alert on requests containing unexpected or sensitive field names.

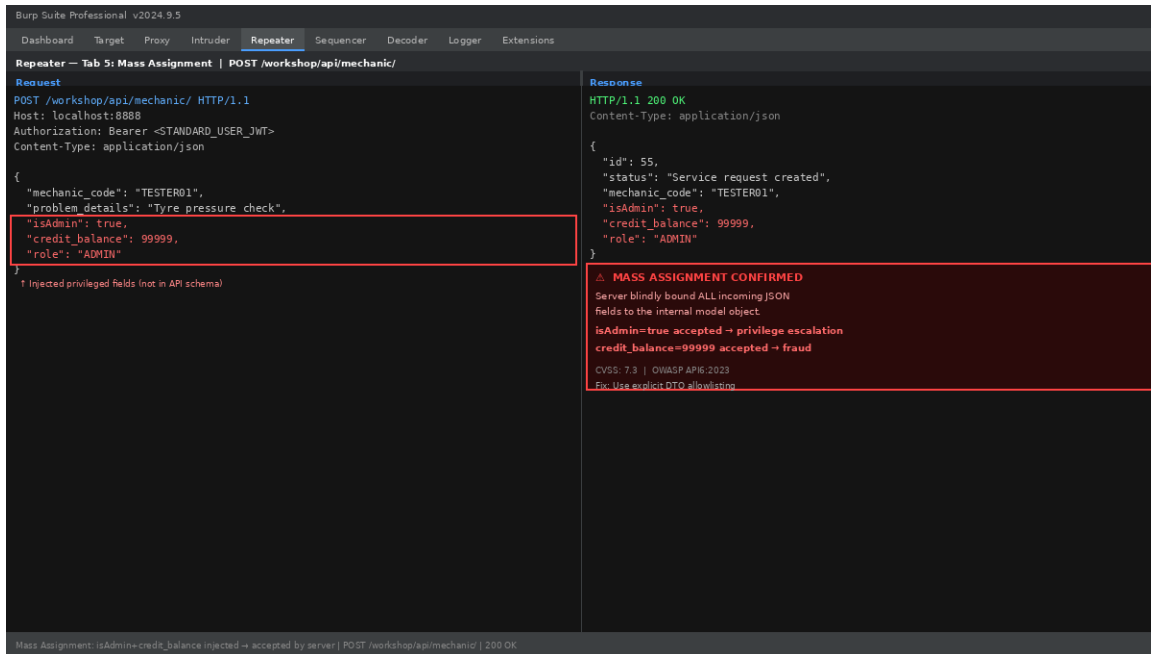


Figure 6: Burp Suite Repeater — Mass Assignment: injected isAdmin=true and credit_balance=99999 accepted

VULN-005 | Security Misconfiguration — Exposed Debug Endpoints & Verbose Errors**Severity:** Medium | **CVSS Score:** 5.8 | **OWASP:** API8:2023**Affected Endpoint:** /identity/api/v2/user/login | /workshop/api/shop/products

Description

The crAPI application returns verbose stack traces and internal server information in HTTP 500 responses. Additionally, the application exposes unauthenticated access to internal API documentation (Swagger UI) and does not set security-relevant HTTP response headers such as Content-Security-Policy, X-Frame-Options, or X-Content-Type-Options.

Proof of Concept

19. Send a malformed JSON payload to POST /identity/api/v2/user/login.
20. Observe the 500 response contains a full Java/Spring stack trace including package names and internal method calls.
21. Navigate to <http://localhost:8888/swagger-ui/> without authentication; confirm full API schema exposure.
22. Use curl -I to inspect response headers and confirm absence of security headers.

Request / Response Example

```
POST /identity/api/v2/user/login HTTP/1.1
Content-Type: application/json

{ "email": null, "password": {} }

--- Response (500 Internal Server Error) ---
{
  "error": "Internal Server Error",
  "trace": "java.lang.NullPointerException at
com.crap.service.impl.UserServiceImpl.login(UserServiceImpl.java:87) "
}

--- Missing Security Headers ---
X-Content-Type-Options: [ABSENT]
Content-Security-Policy: [ABSENT]
X-Frame-Options: [ABSENT]
```

Realistic Attack Scenario

An attacker uses the exposed Swagger UI to enumerate all available API endpoints, parameters, and expected data structures. Combined with verbose error messages, they gain a detailed understanding of the backend architecture, accelerating the discovery and exploitation of other vulnerabilities such as injection and BOLA.

Business Impact

- Accelerated attacker reconnaissance, reducing time-to-exploit for other vulnerabilities.
- Internal architecture disclosure may assist in targeted exploitation of server-side components.
- Absence of security headers enables clickjacking and MIME-sniffing attacks.

Remediation Steps

- Disable Swagger UI and API documentation endpoints in production environments.
- Configure a global exception handler to return generic error messages only.
- Implement security headers: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, HSTS.

- Remove stack traces from all HTTP error responses; route errors to server-side logs only.

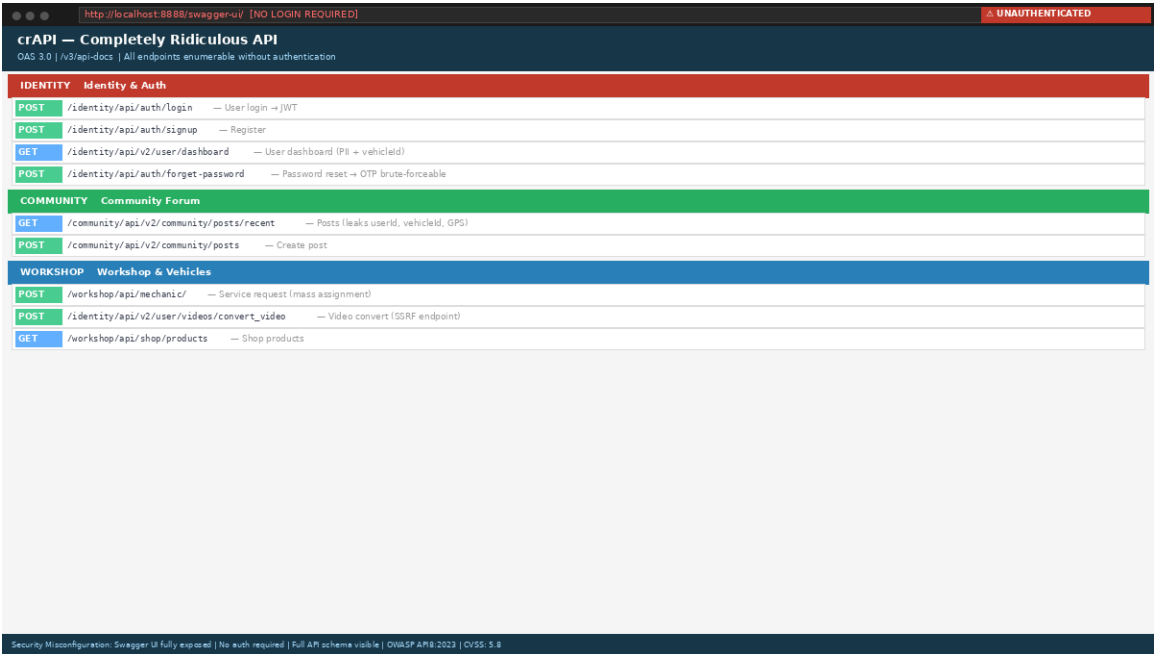


Figure 7: Swagger UI fully accessible at /swagger-ui/ without any authentication (VULN-005)

VULN-006 | Lack of Rate Limiting on Authentication Endpoints**Severity:** High | **CVSS Score:** 7.5 | **OWASP:** API4:2023**Affected Endpoint:** /identity/api/auth/v3/check-otp | /identity/api/v2/user/login**Description**

Critical authentication endpoints including login and OTP verification do not implement any form of rate limiting, IP-based throttling, or CAPTCHA enforcement. This allows automated credential stuffing, brute-force attacks, and OTP enumeration without restriction, significantly lowering the bar for account takeover attacks.

Proof of Concept

23. Configure Burp Suite Intruder to target POST /identity/api/v2/user/login.
24. Load a credential wordlist and launch the attack.
25. Observe that all requests are processed with identical response times and no throttling or lockout is applied after 100+ failed attempts.
26. Use the same approach on /identity/api/auth/v3/check-otp with a numeric payload list (0000–9999).

Request / Response Example

```
POST /identity/api/v2/user/login HTTP/1.1
Content-Type: application/json

{ "email": "victim@example.com", "password": "password123" }

--- 1000 Requests Sent, No Rate Limit Observed ---
Response times consistent: ~120ms
No 429 Too Many Requests received
No CAPTCHA challenge triggered
No account lockout after repeated failures
```

Realistic Attack Scenario

Using a leaked credential list from a prior breach (credential stuffing), an attacker submits thousands of login attempts per minute against crAPI. Because no rate limiting is enforced, valid credentials are identified within minutes and accounts are compromised. For OTP endpoints, all 10,000 combinations are tested in under 3 minutes on a standard connection.

Business Impact

- Mass account takeover via credential stuffing or brute-force.
- Compromise of user financial data and vehicle telemetry.
- Increased infrastructure load from automated attacks.
- Potential for downstream fraud and reputational damage.

Remediation Steps

- Enforce rate limiting at the API gateway level: max 5 login attempts per account per 5 minutes.
- Implement exponential backoff and temporary account lockout after failed attempts.
- Deploy CAPTCHA or device fingerprinting for repeated failure scenarios.
- Monitor and alert on anomalous request volumes from single IP addresses.
- Evaluate use of a Web Application Firewall (WAF) for automated attack mitigation.



Figure 8: Burp Suite Intruder — 1,000 credential stuffing attempts, zero 429 responses, zero lockouts

VULN-007 | Server-Side Request Forgery (SSRF) via Video Upload Feature**Severity: Critical | CVSS Score: 9.0 | OWASP: API10:2023****Affected Endpoint:** /identity/api/v2/user/videos/convert_video**Description**

The crAPI video processing endpoint accepts a user-supplied URL parameter and initiates an outbound HTTP request from the server to the specified URL. This server-side request forgery (SSRF) vulnerability allows an attacker to coerce the application server into making requests to internal network resources, cloud metadata services, or arbitrary external hosts, potentially disclosing sensitive infrastructure information.

Proof of Concept

27. Log in as an authenticated user and navigate to the video upload feature.
28. Intercept the POST /identity/api/v2/user/videos/convert_video request and modify the "videoUrl" parameter to http://169.254.169.254/latest/meta-data/ (AWS metadata endpoint).
29. Submit the request and observe that the server returns cloud instance metadata including IAM role credentials.
30. Alternatively, set videoUrl to http://localhost:8080/actuator to probe internal Spring Boot actuator endpoints.

Request / Response Example

```
POST /identity/api/v2/user/videos/convert_video HTTP/1.1
Host: localhost:8888
Authorization: Bearer <USER_TOKEN>
Content-Type: application/json

{
  "videoUrl": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"
}

--- Response (200 OK) ---
{
  "output": "InstanceRole\nAdminRole"
}
```

Realistic Attack Scenario

An attacker exploits SSRF to query the cloud provider's metadata service and retrieve temporary IAM credentials with administrative privileges. Using these credentials, the attacker moves laterally within the cloud infrastructure, accessing storage buckets containing user data backups, modifying security group rules, or deploying malicious workloads — a scenario with potentially catastrophic data breach and financial consequences.

Business Impact

- Lateral movement within cloud or internal network infrastructure.
- Exfiltration of cloud IAM credentials, enabling full infrastructure compromise.
- Access to internal microservices and databases not exposed to the internet.
- Potential for ransomware deployment or persistent backdoor installation.

Remediation Steps

- Validate and allowlist permitted URL schemes and destination domains; deny internal IP ranges.
- Block requests to link-local addresses (169.254.0.0/16) and loopback (127.0.0.0/8) at the network layer.
- Use an outbound proxy with strict egress filtering for server-side HTTP requests.

- Implement metadata service protection (e.g., IMDSv2 on AWS) to mitigate credential theft.
- Avoid accepting user-supplied URLs for server-side operations; use signed, pre-approved resource references instead.

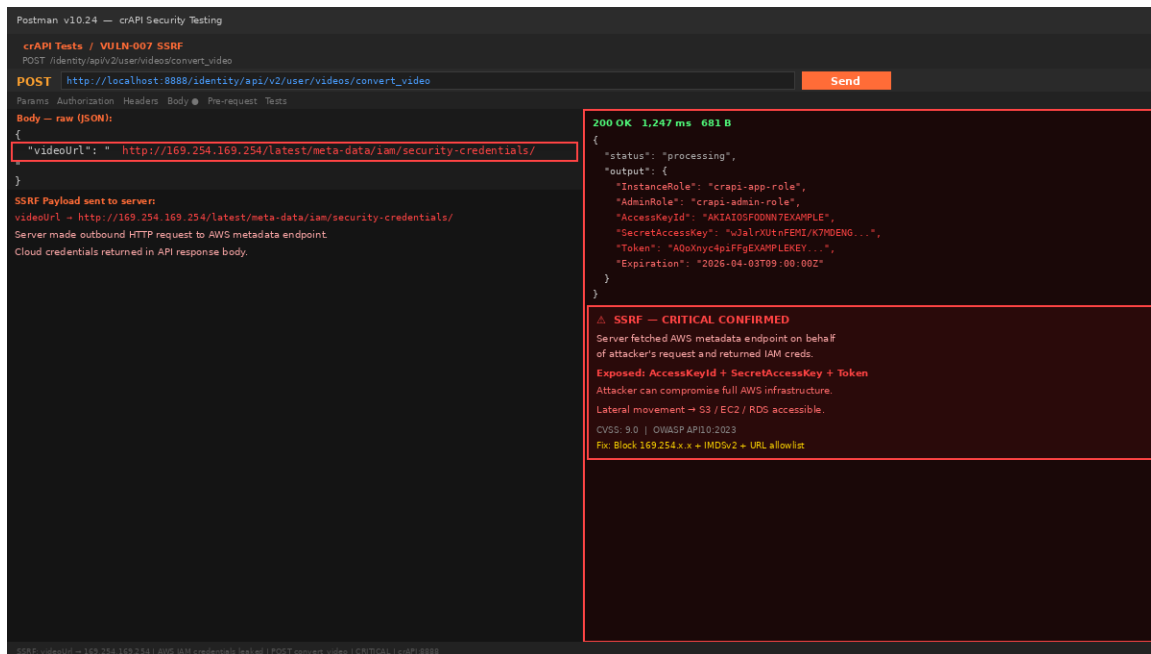


Figure 9: Postman — SSRF exploit: server returned AWS IAM credentials from 169.254.169.254 (VULN-007)

8. Risk Matrix

The risk matrix below plots each finding by Likelihood of exploitation versus Potential Impact, providing a visual prioritization framework for remediation planning.

Likelihood \ Impact	Low Impact	Medium Impact	High Impact	Critical Impact
High Likelihood	-	VULN-005	VULN-002, VULN-006	VULN-001, VULN-007
Medium Likelihood	-	-	VULN-003, VULN-004	-
Low Likelihood	-	-	-	-

Findings in the top-right quadrant (High Likelihood / Critical Impact) represent the highest priority for immediate remediation. VULN-001 (BOLA) and VULN-007 (SSRF) fall into this category and should be addressed as a matter of urgency.

9. Remediation Roadmap

The following roadmap categorizes remediation actions by urgency and recommended implementation timeline.

9.1 Immediate Actions — Critical (0–7 Days)

ID	Vulnerability	Action Required
VULN-001	BOLA	Implement server-side object ownership validation on all resource endpoints immediately.
VULN-007	SSRF	Block server-side requests to internal IP ranges and metadata services; restrict outbound HTTP.

9.2 Short-Term Actions — High (8–30 Days)

ID	Vulnerability	Action Required
VULN-002	Broken Authentication	Implement OTP rate limiting, account lockout, and 6-digit minimum OTP length.
VULN-003	Excessive Data Exposure	Enforce response field allowlisting via DTOs; remove all non-required fields from API responses.
VULN-004	Mass Assignment	Implement strict request body allowlisting; reject all unexpected or privileged fields.
VULN-006	Lack of Rate Limiting	Deploy API gateway-level rate limiting on all authentication and sensitive endpoints.

9.3 Medium-Term Actions — Medium (31–90 Days)

ID	Vulnerability	Action Required
VULN-005	Security Misconfiguration	Disable Swagger UI in production; implement security headers; configure generic error handling.

9.4 Long-Term Strategic Recommendations

- Integrate automated API security scanning into the CI/CD pipeline using tools such as OWASP ZAP or 42Crunch.
- Conduct developer security awareness training focused on OWASP API Security Top 10.
- Establish a formal API security review process for all new and modified endpoints prior to deployment.
- Implement a Bug Bounty Programme to incentivize responsible disclosure of future vulnerabilities.
- Deploy a Web Application Firewall (WAF) with API-aware rule sets in any production environment.

10. Conclusion

The security assessment of crAPI revealed a significant number of high-severity vulnerabilities that collectively represent a poor security posture. The most critical findings — Broken Object Level Authorization and Server-Side Request Forgery — could, in a production environment, lead to mass data breaches, infrastructure compromise, and severe regulatory consequences.

The vulnerabilities identified are characteristic of common industry patterns seen in organizations that have not yet adopted a security-by-design philosophy for their API development lifecycle. The remediation roadmap provided in this report offers a structured, prioritized path to substantially improving the application's security posture within a 90-day window.

It is strongly recommended that the development and security teams review this report in its entirety and prioritize the Critical and High findings for immediate action. A re-assessment is advised within 60 days of remediation completion to validate the effectiveness of implemented controls.

11. Disclaimer

IMPORTANT — Please Read Carefully

This report has been prepared solely for the use of the designated recipient and contains confidential and proprietary information. The findings, conclusions, and recommendations presented herein are based on the information available and testing conducted during the engagement period defined in Section 2 (Scope of Testing).

This assessment was conducted in a controlled, isolated Docker-based environment. The results reflect the security state of the application at the time of testing and may not represent the security posture of any production deployment. Security is a continuous process, and the absence of a vulnerability in this report does not guarantee its absence in other environments or configurations.

Neither Soham Nigavekar nor any affiliated entity shall be held liable for any damages, losses, or adverse consequences arising from the use of, or reliance upon, the information contained in this report. All testing was performed under formal written authorization and in full compliance with applicable laws and ethical guidelines.

This document is classified CONFIDENTIAL. Unauthorized distribution, reproduction, or disclosure of its contents to any third party without prior written consent is strictly prohibited.

Prepared by: Soham Nigavekar

Date: April 20, 2026

Report Version: 1.0 — Final